

An Edge–Cloud RAG Architecture for Real-Time Health Monitoring

Giorgio Acossi¹, Marco Castagna¹, and Giorgio Delzanno¹

DIBRIS . . .

Abstract. This paper presents an IoT health-monitoring platform that connects patients and healthcare professionals through real-time telemetry and AI-assisted interaction. The platform adopts a hybrid architecture: edge services handle data acquisition, validation, caching, and offline operation, while cloud services provide persistent storage, centralized monitoring, and Retrieval-Augmented Generation (RAG). The clinical assistant retrieves relevant passages from a biomedical knowledge base and combines them with patient queries; an additional rule-based workflow enriches this context with labels derived from recent vital-sign measurements. Separate dashboards support patients and clinical experts. The prototype illustrates how RAG can improve the relevance and grounding of responses by supplying the language model with domain-specific evidence, while deterministic analysis provides an explicit link to the patient’s current physiological state. The system is intended as a research prototype and not as a substitute for professional medical assessment.

Keywords: Retrieval-Augmented Generation, edge computing, IoT, health monitoring, large language models

1 Introduction

Retrieval-Augmented Generation (RAG) combines language-model generation with the retrieval of external knowledge. By placing relevant source material in the model context, RAG can produce responses that are more closely grounded in domain-specific information than responses generated from model parameters alone. Most conventional RAG systems, however, operate on static or slowly changing document collections. Health-monitoring applications must also account for a second source of context: physiological measurements that change continuously.

In this setting, information freshness is especially important. A response based only on biomedical literature may be generally relevant yet fail to reflect the patient’s current condition. Conversely, raw sensor values provide timely observations but do not, by themselves, supply the explanatory context needed for a useful answer. Combining these sources requires an architecture that preserves low-latency local operation while supporting centralized retrieval and long-term analysis.

The proposed prototype addresses this problem through a hybrid edge–cloud design. Sensor telemetry is validated and cached at the edge, then forwarded to the cloud for persistence. The cloud hosts a document-based RAG pipeline, while a deterministic diagnosis block summarizes recent measurements and assigns a structured clinical label. The retrieved passages and the label are then provided to the language model as complementary context.

This work examines the architecture and behavior of the resulting system. Its scope is limited to a functional prototype based on simulated sensor data; it does not claim clinical validation or autonomous diagnostic capability.

1.1 Research Questions

- RQ1: How can edge and cloud components be organized to support low-latency monitoring and RAG-assisted interaction?
- RQ2: How can static biomedical knowledge and recent sensor measurements be combined in the context supplied to a language model?
- RQ3: What operational trade-offs emerge between local availability, centralized processing, and response grounding?

1.2 Contributions

This paper makes three main contributions. First, it describes a hybrid IoT architecture that separates latency-sensitive edge functions from storage- and computation-intensive cloud services. Second, it presents two complementary assistant workflows: a conventional document-based RAG pipeline and an enriched pipeline that incorporates a deterministic analysis of recent vital signs. Third, it discusses the operational benefits and limitations observed in a prototype that supports local monitoring during network outages and centralized supervision across multiple edge nodes.

2 Hybrid Architecture Overview

The platform adopts a distributed architecture organized into two tiers: the **Edge Tier**, which prioritizes responsiveness and local availability, and the **Cloud Tier**, which provides centralized storage and computation. The current deployment contains one edge node serving one patient. The design nevertheless allows multiple independent edge nodes to register with the same cloud backend, each monitoring a different patient and forwarding telemetry through a shared gateway. This supports horizontal expansion without changing the main architectural components, although scalability under production workloads remains to be evaluated.

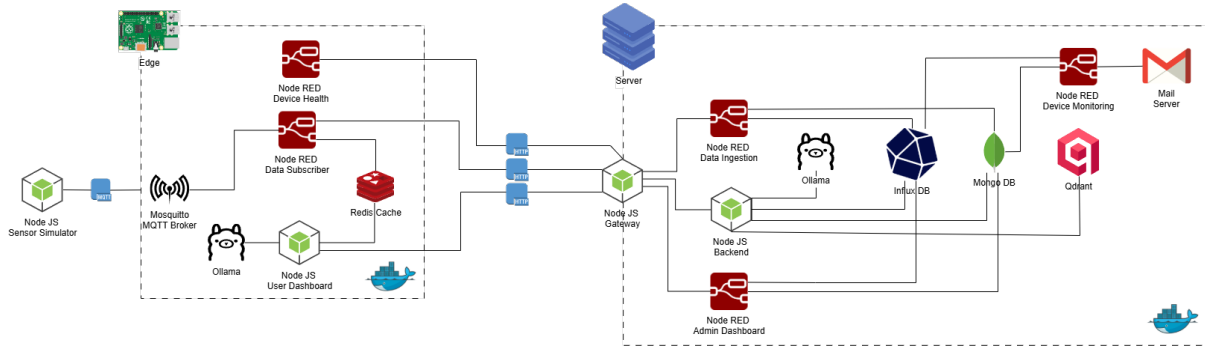


Fig. 1. Hybrid Architecture Overview

3 Edge Tier: Local Processing and Ingestion

The Edge Tier is the platform's first processing layer. It handles high-frequency data acquisition, local validation, and patient interaction. Sensor readings are filtered and cached locally for immediate use while also being forwarded to the cloud. The patient dashboard continues to display locally cached measurements when network connectivity is unavailable. Cloud-dependent RAG responses, however, require connectivity. The tier comprises the following components:

- **Node.js Sensor Simulator:** Generates synthetic vital sign data to model clinical scenarios, including healthy baselines and critical events such as cardiac arrest.
- **Mosquitto MQTT Broker:** Receives data published by the sensor simulator and distributes them to downstream subscribers.
- **Node-RED Device Health Flow:** Monitors the operational status of connected sensors and devices, detecting and reporting faults or connectivity anomalies.
- **Node-RED Data Subscriber Flow:** Subscribes to the MQTT telemetry stream, validates incoming packets, and forwards them both to the local Redis cache and to the cloud gateway.
- **Redis Cache:** Provides state management for vital signs, ensuring that the most recent physiological readings are immediately available to the local dashboard.
- **Node.js User Dashboard:** Displays real-time patient vitals and exposes the AI assistant chat interface. It reads from the local Redis cache so that vital-sign monitoring remains available offline.
- **Local Ollama Instance:** Executes a local LLM at the edge to perform intent detection, acting as a safety gatekeeper that determines whether a user query is clinically relevant before escalating it to the cloud, thereby reducing latency and filtering unsafe or out-of-scope requests.

This arrangement enables continued local monitoring during periods of network unavailability, although cloud-hosted assistant functions are temporarily unavailable.

4 Cloud Tier: Centralized Intelligence

The Cloud Tier hosts the computationally intensive and stateful parts of the platform. It receives validated telemetry from the edge and stores it in specialized databases for trend analysis and clinical review. It also exposes the REST API, executes the complete RAG pipeline, and coordinates device monitoring and alert notifications. The tier comprises the following components:

- **Node.js Gateway:** Serves as the central entry point for edge-to-cloud communication over HTTPS and performs request validation and traffic routing.
- **Node.js Backend:** Implements the application logic and exposes the REST API consumed by the dashboards and AI assistant.
- **Node-RED Data Ingestion:** Standardizes incoming telemetry streams from the edge before committing them to the time-series database.
- **Node-RED Device Monitoring:** Orchestrates the device health-state tracking flow and triggers alert notifications when anomalous conditions are detected.
- **Node-RED Admin Dashboard:** Provides a dedicated monitoring interface for system administrators, enabling real-time visibility into data flows and service health.
- **Ollama Instance:** Hosts the full-capacity LLM for complex clinical reasoning and RAG-based response generation.
- **InfluxDB:** Time-series database for the long-term persistence of high-frequency sensor vitals, enabling trend analysis and historical review by clinical experts.
- **MongoDB:** Document-oriented database for application metadata, user profiles, and clinical session records.
- **Qdrant:** Vector search engine that indexes medical document embeddings, enabling semantic retrieval for the RAG pipeline.

5 System Workflows

The platform coordinates three main workflows across the two tiers: IoT telemetry ingestion, document-based RAG, and RAG enriched with rule-based analysis.

5.1 IoT Data Ingestion Workflow

This workflow transfers sensor telemetry from the edge to the cloud while keeping the latest valid readings available locally.

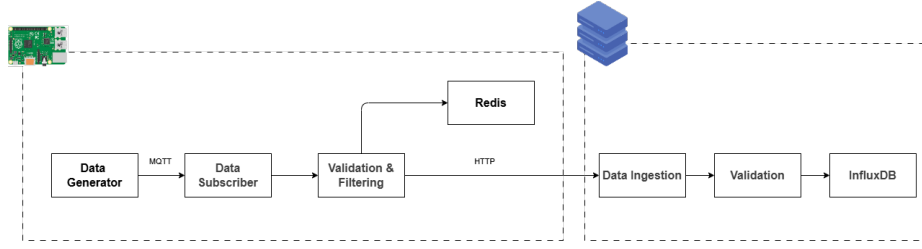


Fig. 2. IoT Data Ingestion Flow

The **Data Generator** publishes simulated measurements over MQTT, and the **Data Subscriber** forwards them to **Validation & Filtering**. Valid readings are cached in **Redis** for local access and sent over HTTP to the cloud. The **Data Ingestion** and **Validation** stages process the incoming telemetry before storing it in **InfluxDB** for historical analysis.

5.2 AI Assistant and RAG Pipeline

This workflow implements a conventional document-based RAG pipeline. Its knowledge base consists of biomedical reference texts indexed in Qdrant. Rather than analyzing live sensor data, it supports patient queries by retrieving relevant passages and supplying them as context to the language model. Queries are screened at the edge for clinical relevance before retrieval and generation take place in the cloud. RAG therefore improves the informational basis of an answer, but does not independently determine the patient’s current clinical state.

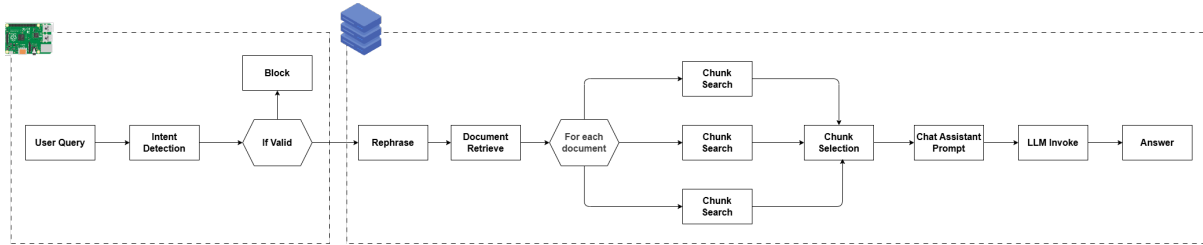


Fig. 3. RAG Pipeline Flow

The pipeline starts when a **User Query** reaches the edge-level **Intent Detection** component. The resulting **If Valid** decision acts as a gate: requests outside the supported clinical scope are sent to **Block**, whereas valid requests are forwarded to the cloud. The **Rephrase** step rewrites the query into a form better suited to semantic retrieval, after which **Document Retrieve** identifies relevant biomedical documents in Qdrant. The **For each document** stage

processes every retrieved document through **Chunk Search**; the resulting passages converge at **Chunk Selection**, which retains the most relevant context. The **Chat Assistant Prompt** then combines these passages with the original user request and the assistant instructions. Finally, **LLM Invoke** generates the grounded **Answer** returned to the patient dashboard. This workflow uses the indexed biomedical corpus but does not incorporate live sensor readings; retrieval improves the informational basis of the response without establishing the patient’s current clinical state.

5.3 Rule-Based Diagnosis Workflow

This workflow extends the document-based RAG pipeline with a deterministic analysis of recent telemetry. Measurements from a configurable trailing window, set to five minutes in the prototype, are summarized and compared with predefined clinical thresholds. The resulting label is combined with retrieved biomedical passages, allowing the model to consider both general medical information and a concise representation of the patient’s recent state. These labels serve only as decision-support inputs within the prototype and are not clinically validated diagnoses.

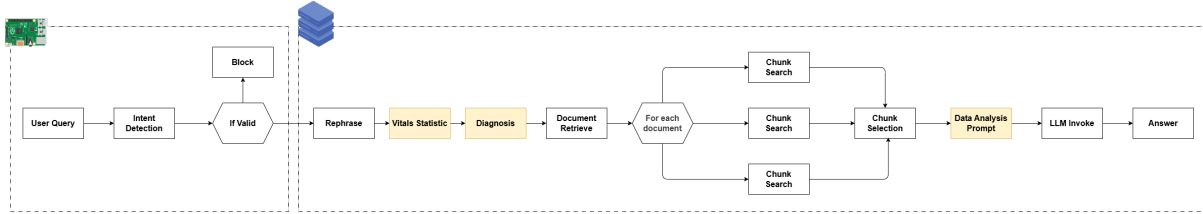


Fig. 4. Data Analyst and Rule-Based Diagnosis Flow

The rule-based workflow retains the same initial control path: **User Query**, **Intent Detection**, and **If Valid** either reject an unsupported request through **Block** or pass it to **Rephrase**. It then introduces **Vitals Statistic**, which retrieves measurements from the preceding five minutes and computes the minimum, maximum, and mean for each metric. The **Diagnosis** step compares these aggregated values with configured thresholds and produces a structured clinical label, such as *Tachyarrhythmia* or *Hypoxemia*. Processing then continues with **Document Retrieve**; **For each document** initiates the corresponding **Chunk Search** operations, and **Chunk Selection** chooses the passages most relevant to the query. Unlike the standard assistant pipeline, **Data Analysis Prompt** combines the selected biomedical passages with the rule-based label and recent vital-sign statistics. **LLM Invoke** uses this enriched context to produce the final **Answer**. The label is a decision-support input generated by the prototype and should not be interpreted as a clinically validated diagnosis.

6 Clinical Dashboard and Simulation Analysis

The platform provides two interfaces for distinct roles: a patient dashboard at the edge and an expert dashboard in the cloud.

6.1 Patient Dashboard

The Patient Dashboard is the primary interface for the monitored patient. It reads vital signs directly from the local Redis cache and displays heart rate, blood pressure, oxygen saturation, and respiratory rate even when cloud connectivity is lost. When the cloud is reachable, the same interface provides access to the AI assistant and its RAG-generated responses.

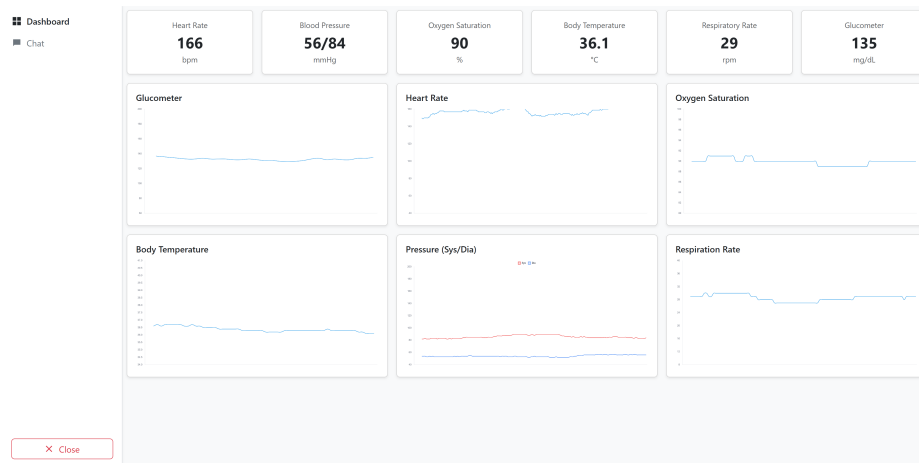


Fig. 5. Patient Dashboard: Vitals Visualization

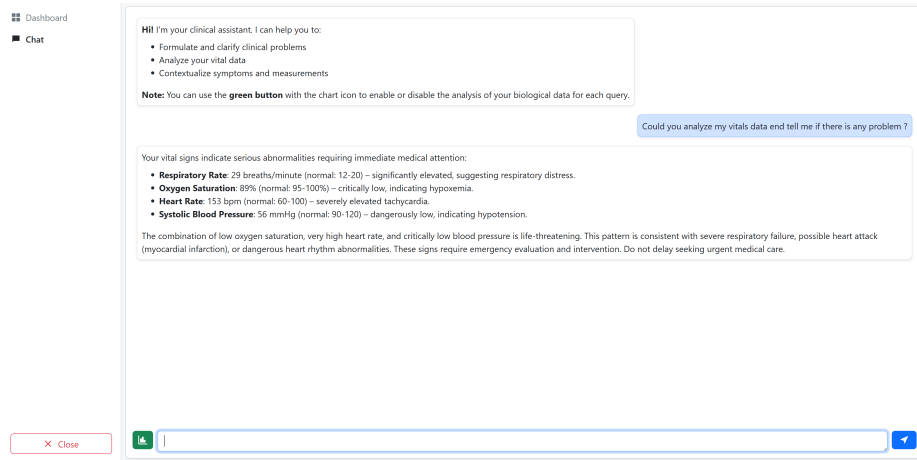


Fig. 6. Patient Dashboard: AI Assistant Chat Interface

6.2 Expert Dashboard

The Expert Dashboard supports healthcare professionals and system administrators. It contains a device panel showing registered edge nodes and their connectivity status, a panel for active and historical monitoring sessions, and a detailed view with live vital signs and interaction history for a selected patient. Together, these views provide a centralized overview of multiple edge nodes and highlight conditions flagged by the monitoring logic.

Devices			
Search			
Device Id	Name	Status	Last Update
device-1	Device-1	●	2025-12-02T22:29:30.981Z
device-2	Device-2	●	-
device-3	Device-3	●	-
device-4	Device-4	●	-

Fig. 7. Expert Dashboard: Devices Telemetry

Sessions				
Search				
Id	Device Id	Created At	Updated At	Status
692f541f7676d34c8d7b87	device-1	2025-12-02T21:03:27.201Z	2025-12-02T21:03:27.201Z	●
692f53e17676d34c8d7b85	device-1	2025-12-02T21:02:25.433Z	2025-12-02T21:03:25.870Z	●
692f126c3ab92063c48e2507	device-1	2025-12-02T16:23:08.460Z	2025-12-02T16:27:37.269Z	●
692f10233ab92063c48e2506	device-1	2025-12-02T16:13:23.677Z	2025-12-02T16:23:06.825Z	●
692f0b423ab92063c48e2505	device-1	2025-12-02T15:52:34.336Z	2025-12-02T16:13:23.674Z	●
692f0b273ab92063c48e2504	device-1	2025-12-02T15:52:07.228Z	2025-12-02T15:52:33.436Z	●
692ec61f505b94edfe6651c9	device-1	2025-12-02T10:57:35.199Z	2025-12-02T15:52:07.223Z	●
692ec5aa505b94edfe6651c7	device-1	2025-12-02T10:55:38.849Z	2025-12-02T10:57:34.354Z	●
692ec4cd505b94edfe6651c6	device-1	2025-12-02T10:51:57.716Z	2025-12-02T10:55:38.008Z	●
692ec3fb505b94edfe6651c4	device-1	2025-12-02T10:48:27.131Z	2025-12-02T10:51:56.577Z	●
692e1535a252cd39bc591bea	device-1	2025-12-01T22:22:45.094Z	2025-12-02T10:48:27.128Z	●
692e11ff386e1f7db71b6970	device-1	2025-12-01T22:09:03.277Z	2025-12-01T22:22:43.804Z	●
692e0f89ca34c424ef51e6d6	device-1	2025-12-01T21:58:33.346Z	2025-12-01T22:09:01.701Z	●
692e0e83ce34c424ef51e6d3	device-1	2025-12-01T21:54:11.780Z	2025-12-01T21:58:32.045Z	●
692e0d60ee85703df949192	device-1	2025-12-01T21:49:20.887Z	2025-12-01T21:54:10.457Z	●
692e0bbcf8f237ad81a320d	device-1	2025-12-01T21:42:23.084Z	2025-12-01T21:49:19.343Z	●
692e089e0d78fac9219711e	device-1	2025-12-01T21:29:02.342Z	2025-12-01T21:42:21.678Z	●

Fig. 8. Expert Dashboard: Active Sessions List

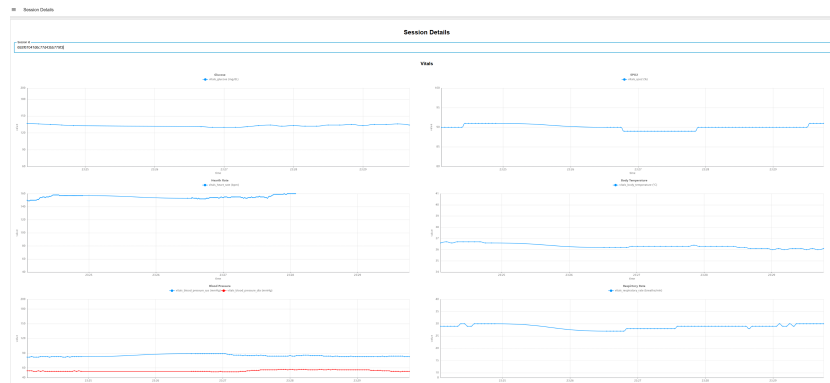


Fig. 9. Expert Dashboard: Detailed Session Vitals

7 Future Developments

The current implementation provides a functional prototype that can be extended in several directions. Future developments will focus on improving the clinical specialization of the AI components, integrating the platform with real devices, and strengthening its deployment architecture. The main areas under consideration are the following:

- **LLM Fine-Tuning:** Adapt the language models to the medical domain to improve terminology, response relevance, and consistency.
- **Graph RAG:** Introduce knowledge graphs to represent and retrieve complex relationships between symptoms, conditions, measurements, and treatments.
- **Autonomous Agents:** Explore agents capable of performing controlled multi-step reasoning and assisting with predefined monitoring tasks.
- **Machine-Learning-Based Diagnosis:** Extend the current deterministic rule engine with predictive models, while retaining interpretable constraints and human oversight.
- **Real-Device Integration:** Replace the sensor simulator with interfaces for real medical and wearable devices.
- **Clinical Validation:** Conduct supervised studies with healthcare professionals to evaluate response quality and system behavior in realistic scenarios.

8 Conclusion

This paper presented a hybrid edge–cloud platform for real-time health monitoring and AI-assisted patient interaction. The edge tier validates and caches telemetry, supports continuous local visualization, and screens user queries. The cloud tier stores historical data, provides centralized supervision, and executes two assistant workflows: document-based RAG and an enriched variant that also incorporates rule-based summaries of recent vital signs.

The prototype indicates that retrieval can improve response quality by giving the language model relevant biomedical context instead of relying exclusively on knowledge encoded in its parameters. In the enriched workflow, the deterministic diagnosis block further aligns the response with recent sensor observations. The principal benefit of RAG in this system is therefore not autonomous diagnosis, but better-grounded and more contextually relevant communication. This distinction is particularly important in a health-monitoring setting, where fluent output alone is insufficient evidence of correctness.

The work also identifies clear limits. The current implementation relies on simulated data and qualitative observations, and it has not undergone clinical validation or a controlled evaluation of response quality. The results should consequently be interpreted as evidence of architectural feasibility rather than clinical effectiveness. Future experiments should compare RAG and non-RAG

responses, assess source attribution and factual consistency, measure end-to-end latency, and involve qualified clinical reviewers. Within these boundaries, the architecture provides a practical basis for studying how retrieval, streaming telemetry, and deterministic analysis can complement one another in remote health-assistance systems.